

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I ____Delisha A K (192511307)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Smart Banking System

In a Smart Banking System, **BankAccount** is a class containing attributes such as account number, customer name, and balance, and methods like **deposit()**, **withdraw()**, and **transfer()**. | Classes promote code reuse and simplify software development. | A customer's savings account is an object of the **BankAccount** class with its own account number and balance.

(a) Explain the following concepts

1. Class

A **Class** is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) of an object.

Example: In a Smart Banking System, **BankAccount** is a class containing attributes such as account number, customer name, and balance, and methods like **deposit()**, **withdraw()**, and **transfer()**.

Application: Classes promote code reuse and simplify software development.

2. Object

An **Object** is an instance of a class that represents a real-world entity and stores actual data.

Example: A customer's savings account is an object of the **BankAccount** class with its own account number and balance.

Application: Objects allow the system to manage multiple customer accounts independently.

3. Message Passing

Message Passing is the communication between objects through method calls.

Example: When a customer transfers money, the **Customer** object sends a request to the **BankAccount** object to execute the **transfer()** method.

Application: It enables secure and efficient interaction between different banking objects.

4. Encapsulation

Encapsulation is the process of combining data and methods into a single unit while restricting direct access to sensitive data.

Example: The account balance is declared private and can only be accessed through methods such as **deposit()** and **withdraw()**.

Application: It improves security, protects customer information, and prevents unauthorized modifications.

5. Polymorphism

Polymorphism allows the same method to perform different operations depending on the object.

Example: The **calculateInterest()** method calculates interest differently for Savings, Current, and Fixed Deposit accounts.

Application: It reduces duplicate code and makes the banking system flexible and easy to extend.

(b) Describe the following relationships

1. Association

Association is a relationship where two objects are connected but can exist independently.

Example: A **Customer** is associated with one or more **Bank Accounts**.

Application: It represents the relationship between customers and their accounts without creating dependency.

2. Aggregation

Aggregation is a weak "has-a" relationship where one object contains another, but both can exist independently.

Example: A **Bank** has many **Customers**, but customers can exist even if they close their accounts.

Application: It organizes related objects while maintaining their independence.

3. Composition

Composition is a strong "has-a" relationship where the child object cannot exist without the parent object.

Example: A **Bank Account** contains **Transaction** records. If the account is deleted, its transaction history is also removed.

Application: It ensures strong ownership and maintains data consistency.

(c) Explain how these concepts improve software reusability and maintainability in a banking system

Object-Oriented Programming concepts improve the Smart Banking System in many ways:

-
- **Classes** allow developers to reuse code for different account types, reducing development time.
 - **Objects** represent individual customer accounts, making data management efficient.
 - **Message Passing** enables secure communication between system components.
 - **Encapsulation** protects confidential customer information and improves data security.
 - **Polymorphism** allows new account types and services to be added without changing existing code.
 - **Association, Aggregation, and Composition** organize relationships between banking entities, making the system easier to understand and maintain.

These concepts make the Smart Banking System **secure, reusable, scalable, flexible, and easy to maintain**, while supporting future enhancements with minimal changes.

Conclusion

Object-Oriented Programming concepts such as **Class, Object, Message Passing, Encapsulation, Polymorphism, Association, Aggregation, and Composition** help build a Smart Banking System that is efficient, secure, reusable, and maintainable. They improve software quality, reduce development effort, and make future updates easier.

Question 2: Smart Healthcare Management System

(a) Object-Oriented Design Principles

1. Abstraction

Abstraction is the process of hiding unnecessary implementation details and showing only the essential features of an object.

Example: In a Smart Healthcare Management System, a patient can book an appointment without knowing how the database stores the appointment details.

Application: Abstraction simplifies the system, improves security, and allows users to focus only on the required functions.

2. Encapsulation

Encapsulation is the process of combining data and methods into a single unit while protecting data from unauthorized access.

Example: A patient's medical records are kept private and can only be accessed through authorized methods by doctors or hospital staff.

Application: It improves data security, prevents unauthorized modifications, and maintains patient confidentiality.

3. Modularity

Modularity means dividing a software system into smaller independent modules that perform specific tasks.

Example: The healthcare system can have separate modules for Patient Management, Doctor Management, Appointment Booking, Billing, and Pharmacy.

Application: If one module requires modification, the remaining modules continue to function without interruption.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that a subclass should be able to replace its parent class without changing the correctness of the program.

Example: If **Doctor** is the parent class and **Cardiologist** is the child class, replacing a Doctor object with a Cardiologist object should not affect the appointment booking process.

Application: LSP improves software reliability, code reusability, and reduces errors during system development.

2. Interface Segregation Principle (ISP)

The Interface Segregation Principle states that a class should not be forced to implement methods it does not require.

Example: A Doctor interface contains methods related to patient diagnosis, while a Receptionist interface contains methods related to appointment scheduling. Each user accesses only the required functions.

Application: ISP reduces unnecessary dependencies, simplifies maintenance, and improves system flexibility.

(c) How These Principles Improve Software Quality and Flexibility

The object-oriented design principles improve the Smart Healthcare Management System in the following ways:

- **Abstraction** simplifies the system by displaying only necessary features to users.
- **Encapsulation** protects sensitive patient information and improves data security.

-
- **Modularity** divides the software into independent modules, making development and maintenance easier.
 - **LSP** ensures that new specialized classes can be added without affecting existing functionalities.
 - **ISP** allows each user to access only the features relevant to their role.

Together, these principles make the healthcare system **secure, reusable, scalable, flexible, and easy to maintain**, resulting in better software quality and improved patient services.

Conclusion

Object-oriented design principles and SOLID principles help develop a reliable and efficient Smart Healthcare Management System. They improve software quality by increasing security, flexibility, maintainability, and scalability while making future enhancements easier.

Question 3: Online Food Delivery System

(a) Phases of the Software Development Life Cycle (SDLC)

1. Planning

Planning is the first phase of SDLC where project objectives, scope, budget, timeline, and resources are identified.

Example: The company decides to develop an Online Food Delivery System that allows customers to order food, track deliveries, and make online payments.

Purpose: It helps define project goals and ensures proper planning before development begins.

2. Requirement Analysis

Requirement Analysis is the process of collecting, analyzing, and documenting user requirements.

Example: The system should allow customer registration, restaurant management, food ordering, payment, order tracking, and delivery management.

Purpose: It ensures that the software meets customer needs and reduces future changes.

3. Design

The design phase creates the architecture and blueprint of the software.

Example: Designers prepare UML diagrams, database structure, user interface, and system architecture for the Online Food Delivery System.

Purpose: It provides a clear plan for developers to implement the software efficiently.

4. Development

Development is the phase where programmers write the actual source code based on the design.

Example: Developers create modules for login, restaurant management, menu display, cart, payment gateway, and order tracking.

Purpose: It converts the design into a working software application.

5. Testing

Testing checks whether the software works correctly and identifies defects before deployment.

Example: Testers verify login, food ordering, payment processing, order tracking, and notification features.

Purpose: It improves software quality by detecting and fixing errors before users access the system.

6. Maintenance

Maintenance is the final SDLC phase where the software is updated and improved after deployment.

Example: Adding new payment methods, improving performance, fixing bugs, or introducing new delivery features.

Purpose: It ensures the software remains reliable, secure, and meets changing customer requirements.

(b) Purpose of Each SDLC Phase

SDLC Phase	Purpose
Planning	Defines project goals, budget, schedule, and resources.
Requirement Analysis	Collects and documents user requirements accurately.
Design	Creates the software architecture and system design.
Development	Converts the design into a working application through coding.
Testing	Identifies and removes defects to improve software quality.
Maintenance	Updates, improves, and supports the software after deployment.

(c) How SDLC Contributes to Delivering Reliable Software

The Software Development Life Cycle helps deliver reliable software by following a structured development process.

-
- **Planning** ensures clear project objectives and proper resource allocation.
 - **Requirement Analysis** ensures the software satisfies customer needs.
 - **Design** provides a strong architecture, reducing development errors.
 - **Development** converts the design into a functional application.
 - **Testing** detects and removes bugs before software release.
 - **Maintenance** keeps the software updated, secure, and efficient after deployment.

By following all SDLC phases, the Online Food Delivery System becomes **reliable, scalable, maintainable, user-friendly, and capable of meeting future business requirements.**

Conclusion The Software Development Life Cycle (SDLC) provides a systematic approach to software development. Each phase has a specific purpose and contributes to building a reliable, high-quality Online Food Delivery System. Following SDLC reduces risks, improves software quality, and ensures customer satisfaction.

Question 4: Smart Agriculture Monitoring System

(a) SDLC Models

1. Prototype Model

The Prototype Model develops a working prototype before building the final software. Users test the prototype and provide feedback, which is used to improve the system.

Example: Farmers test a prototype of the Smart Agriculture Monitoring System and suggest features like weather alerts and soil moisture monitoring before the final system is developed.

Application: This model helps developers understand changing customer requirements and reduces development risks.

2. Spiral Model

The Spiral Model combines software development with continuous risk analysis. Development is carried out in repeated cycles, where each cycle includes planning, risk analysis, development, and evaluation.

Example: Before adding a new irrigation control feature, developers analyze possible risks such as sensor failures and system errors.

Application: It is suitable for large and complex projects where risk management is important.

3. Agile Model

The Agile Model develops software in small iterations called sprints. Customer feedback is collected after every iteration, and improvements are made continuously.

Example: Farmers request a crop disease prediction feature after the first version. Developers include it in the next sprint without affecting the existing system.

Application: Agile supports changing requirements and delivers software faster.

4. DevOps Model

The DevOps Model integrates software development and operations to enable continuous development, testing, deployment, and monitoring.

Example: After adding a new weather prediction feature, developers deploy updates quickly while continuously monitoring system performance.

Application: It ensures faster software delivery, better collaboration, and reliable system maintenance.

(b) Comparison Between Prototype Model and Agile Model

Prototype Model	Agile Model
Builds a prototype before the final system.	Develops software in small iterations (sprints).
Mainly used to understand user requirements.	Used for continuous development and improvement.
Customer feedback is collected mainly during prototype evaluation.	Customer feedback is collected after every sprint.
Suitable when requirements are unclear.	Suitable when requirements change frequently.
Final product is developed after prototype approval.	Working software is delivered regularly throughout development.

(c) Recommended SDLC Model with Justification

The **Agile Model** is the most suitable for the Smart Agriculture Monitoring System because customer requirements change regularly.

Justification:

- Farmers can suggest new features at any stage.
- New technologies and farming methods can be added easily.
- Frequent updates improve system performance.
- Customer feedback is incorporated continuously.
- Development is faster through small iterations.

-
- The software remains flexible, scalable, and easy to maintain.

Therefore, Agile helps developers quickly respond to changing agricultural needs while maintaining software quality.

Conclusion

The Prototype, Spiral, Agile, and DevOps models each have unique advantages. However, because customer requirements change frequently in a Smart Agriculture Monitoring System, the **Agile Model** is the best choice. It provides flexibility, continuous customer involvement, faster delivery, and high-quality software.

Example: A bus ticket changes through the following states:

Booked → Confirmed → Cancelled → Completed

Application: It helps developers understand how the ticket booking process behaves under different conditions.

Question 5: Smart Transport Management System

(a) Importance of UML in Object-Oriented Software Development

Unified Modeling Language (UML) is a standard visual language used to design, visualize, document, and understand software systems before implementation.

Importance of UML:

- Provides a clear blueprint of the software system.
- Improves communication among developers, designers, and stakeholders.
- Helps identify design errors before coding begins.
- Simplifies the development and maintenance process.
- Makes the software more organized and easier to understand.

Example: In a Smart Transport Management System, UML diagrams help model vehicle tracking, route management, ticket booking, and passenger information before development starts.

(b) UML Diagrams

1. State Diagram

A **State Diagram** represents the different states of an object and the transitions between those states during its lifecycle.

Example: A bus ticket can move through the following states:

- Available
- Booked
- Confirmed
- Cancelled

Application: It helps developers understand how an object behaves during different stages.

2. Component Diagram

A **Component Diagram** shows the different software components and how they interact with each other.

Example: The Smart Transport Management System may contain:

- User Management Component
- Ticket Booking Component
- Payment Component
- Vehicle Tracking Component
- Notification Component

Application: It improves system organization and simplifies module integration.

3. Deployment Diagram

A **Deployment Diagram** shows the physical deployment of hardware and software components.

Example:

- Client Device (Mobile/Web)
- Application Server
- Database Server
- GPS Tracking Server

Application: It helps developers understand how the system is deployed and communicates across devices.

4. Package Diagram

A **Package Diagram** organizes related classes into packages.

Example:

- Passenger Package

-
- Driver Package
 - Booking Package
 - Payment Package
 - Vehicle Package

Application: It improves software organization and makes maintenance easier.

(c) How UML Models Improve Communication Among Stakeholders

UML diagrams improve communication by providing a visual representation of the system that is easy to understand.

They help:

- Developers understand system design before coding.
- Designers create an organized software architecture.
- Clients visualize how the system will function
- Testers identify system modules for testing.
- Project managers monitor project progress effectively.

As a result, UML reduces misunderstandings, improves collaboration, minimizes development errors, and increases software quality.

Conclusion

UML is an essential tool in object-oriented software development. By using **State, Component, Deployment, and Package Diagrams**, developers can design the Smart Transport Management System more effectively. UML improves communication, reduces design errors, and supports the development of reliable, scalable, and maintainable software.

Question 6: Smart Inventory Management System

(a) Concept of Modular Software Design

Modular Software Design is a software development approach in which a large system is divided into smaller, independent modules. Each module performs a specific function and can be developed, tested, maintained, and updated separately.

Example: In a Smart Inventory Management System, the software can be divided into the following modules:

- Product Management
- Supplier Management

- Inventory Tracking
- Order Management
- Billing and Reports

Advantages:

- Simplifies software development.
- Makes testing and debugging easier.
- Allows independent modification of modules.
- Improves software maintenance and reusability.
- Reduces development time and cost.

(b) Comparison Between Procedural Programming and Object-Oriented Programming

Procedural Programming	Object-Oriented Programming (OOP)
Program is divided into functions.	Program is divided into classes and objects.
Focuses on procedures or functions.	Focuses on objects and data.
Data is less secure because it is shared globally.	Data is protected using encapsulation.
Code reusability is limited.	High code reusability through inheritance and polymorphism.
Difficult to maintain large applications.	Easier to maintain and extend large applications.

Application:

For a Smart Inventory Management System, **Object-Oriented Programming** is more suitable because it provides better security, modularity, code reuse, and easier maintenance.

(c) How Modularity Improves Scalability, Maintainability, and Reusability

Modularity provides several benefits to the Smart Inventory Management System:

Scalability

- New modules such as Barcode Scanning or Warehouse Management can be added without affecting existing modules.
- The system can easily grow as business requirements increase.

Maintainability

- Errors can be identified and fixed within a single module.
- Updating one module does not affect the entire software system.

-
- Maintenance becomes faster and less expensive.

Reusability

- Existing modules can be reused in other inventory or business management systems.
- Reusable code reduces development effort and improves software q

Application:

If the company wants to introduce an **AI-based Stock Prediction Module**, it can be added as a separate module without changing the existing Inventory Management, Billing, or Supplier modules.

Conclusion

Modular Software Design and Object-Oriented Programming help develop a **secure, scalable, reusable, and maintainable** Smart Inventory Management System. By dividing the system into independent modules and using OOP principles, software becomes easier to develop, test, maintain, and enhance. This approach improves software quality and supports future business growth.

OOAD



Keep the Winning Streak Alive, DELISHA!

28260

Score

+474

Coins



Shop

Accuracy ⓘ



Performance Stats

45 questions

✓ 32 Correct

✓ 1 Partially correct

✗ 10 Incorrect

⌚ 24 s time/question

🔍 2 Unattempted